

Nouvelle formulation d'une proposition de classification non-supervisée des images hyperspectrales

Gabriel Dauphin

June 3, 2026

1 Introduction

L'idée générale de la proposition inclut un algorithme de classification non-supervisé, un ensemble de paramètres et une mesure de la vraisemblance de la classification obtenue.

L'algorithme de classification non-supervisé peut effectivement être un **kmeans**, un **region-growing**, mais il peut être une classification supervisée et dans ce cas le choix des pixels de la base d'entraînement fait parti de l'ensemble de paramètres choisi.

L'ensemble de paramètres détermine la façon dont l'algorithme de classification non-supervisé effectue son action. Le pari est que pour un algorithme de classification non-supervisé, il existe un choix de valeurs de paramètres qui lorsque utilisé pour une image donnée, il permettent d'obtenir une classification pertinente.

La mesure de vraisemblance permet d'utiliser une connaissance extérieure à l'image, à savoir que les pixels d'une même classe correspondent a priori à un petit nombre de régions avec des contours particuliers. Cette mesure est a priori un nombre que l'on cherche par exemple à maximiser.

Ce découpage en trois parties est utile pour faire des choix, mais la réalité est plus complexe, parce que certains algorithmes se donnent des objectifs similaires à la mesure de vraisemblance (je pense au region-growing), le choix de l'ensemble des paramètres dépend en parti du choix de l'algorithme, et il existe des façons de choisir les paramètres qui permettent de réduire le temps de calcul en éliminant d'office des valeurs de ces paramètres qui donneraient une mesure de vraisemblance mauvaise.

2 Suggestion pour l'ensemble des paramètres

Ce que j'entends par la notion d'ensemble des paramètres est bien un ensemble de valeurs qui peut donc se mettre sous la forme d'un vecteur. Mais cela ne préjuge pas du tout de la façon dont ces valeurs peuvent modifier le fonctionnement de l'algorithme. Voici quelques idées. Je note $I_{m,n,k}$ l'intensité du pixel localisé à la ligne m , à la colonne n et pour la longueur d'onde numéro k

- Les valeurs du paramètres peuvent être utilisés pour être multipliés à l'intensité associée à chaque longueur d'onde. Au lieu d'utiliser $I_{m,n,k}$, j'utilise par exemple dans l'algorithme de classification non-supervisé $c_k I_{m,n,k}$ et $c_0 \dots c_{K-1}$ sont les valeurs de mon ensemble de paramètres composé de K valeurs. Je pourrai aussi choisir que c_k vaut soit 1 soit 0 de façon à ce que mon ensemble de paramètres détermine quelles longueurs d'onde je choisis d'utiliser.
- Je pourrai aussi choisir d'accorder plus d'importance à certains pixels. Ainsi au lieu d'utiliser $I_{m,n,k}$, j'utilise $w_{m,n} I_{m,n,k}$. $w_{m,n}$ étant alors un certain nombre de valeurs positives réelles ou juste de 1 et des 0. L'algorithme de classification supervisé produit alors des clusters et je réinsère les pixels pour lesquels $w_{m,n}$ sont nuls à la fin en les affectant aux clusters qui les ressemblent le plus.
- Dans la présentation orale, j'ai proposé d'utiliser comme paramètre le fait que tel ou tel pixel fait parti d'une classe ce qui permet d'utiliser un algorithme de classification supervisée.
- Dans le document écrit de novembre 2023 (**Towards a proposition for the nsupervised classification of hyperspectral images**), j'avais proposé en 7.5.1, d'utiliser un simple seuil sur une valeur particulière de l'intensité à une longueur d'onde donnée pour déterminer si le pixel appartient à une région ou non.

$$\text{Le pixel } (m, n) \text{ appartient à la classe 1 si } o_k I_{m,n,k} \geq s_k o_k \quad (1)$$

Dans cette formulation, l'ensemble des paramètres est $o_0 \dots o_{K-1}, s_0 \dots s_{K-1}$. Cette technique est appelée un seuillage. Les K premières valeurs sont soit -1 soit 1 . Les structures présentées en 7.3 et 7.4 indiquent que les paramètres peuvent combiner différents seuillages, dans ce cas les paramètres indiquent le nombre, l'ordre et la façon dont les seuillages sont combinés suivant une règle définie par une convention à implémenter, le choix de la convention n'est en fait pas vraiment important, ce qui compte c'est de s'y tenir.

Ce que j'attends du choix de la signification de l'ensemble des paramètres est surtout la possibilité de modifier de façon la plus diverse le fonctionnement de l'algorithme de base, il n'est pas essentiel que cette façon de modifier conduise à de bons résultats puisque ceci est le rôle de la mesure de vraisemblance.

3 Suggestion pour la mesure de vraisemblance

En ce qui la mesure de vraisemblance, le choix effectué a un rôle important dans le succès de la technique. Et l'idée n'est pas de choisir cette mesure de façon à finalement obtenir un OA très proche du ground truth, car ceci serait faire une classification supervisée. Ainsi le choix est crucial, mais il est faux de croire qu'il n'y a qu'un seul choix possible, et est-ce vraiment pertinent de croire qu'il y en a un meilleur. Le premier aspect de la discussion est de considérer des briques de bases, chaque brique mesurant un aspect de ce que l'on pourrait considérer comme une "bonne" classification.

- Le CIP défini en 8.1.1 compte le nombre de points isolés, c'est-à-dire le nombre de pixel dont tous ses voisins (par exemple les 8 pixels en haut, en bas, à gauche, à droite et dans les 4 diagonales) ne seraient pas associés à la même classe. Dans l'équation (46), m désigne non pas le numéro de ligne mais le numéro du pixel dans l'image, c'est juste une manière d'alléger les notations. Dans ce cas on cherche à minimiser le CIP.
- Le CE défini en 8.1.2 compte le nombre de pixels appartenant à un contour, c'est-à-dire le nombre de pixels dont un des voisins ne fait pas parti de la même classe. Ici on cherche aussi à ce que CE ne soit pas trop grand.
- Le CNCR compte le nombre de régions connectés. Ce qu'on appelle région connexe est défini dans https://en.wikipedia.org/wiki/Pixel_connectivity Une région est connexe, s'il existe une succession de pixels permettant d'aller d'un pixel de cette région à un autre en étant toujours proche d'un voisin. Concrètement il existe une implémentation expliquée par exemple dans https://www.tutorialspoint.com/scipy/scipy_connected_component_labeling.htm L'algorithme 5 décrit une façon d'implémenter le fait de trouver ces régions connexes. Le CNCR est la somme des nombre de régions connexes par les pixels pour chaque classe.
- Le CPISR généralise le CIP en incluant les régions composées non pas d'un seul pixel mais d'un petit nombre de pixels.
- AVG_RND, AVG_SC, MED_LSL sont des indicateurs la rondeur et l'étalement des régions.
- Je rajoute ici une nouvelle idée, celle de vérifier si les contours ont tendance à former des lignes droites.
 1. Transformer l'image des classes en une image de contours. Comme le décrit l'équation (47), un pixel fait parti d'un contour si un de ses voisins ne fait pas parti de la même classe.
 2. On tourne l'image des contours d'un angle θ . Par exemple avec <https://fr.moonbooks.org/Articles/Comment-tourner-une-image-en-python-avec-le-module-pillow/>
 3. On considère chaque pixel de l'image et pour chacun, on comptabilise le nombre de déplacements verticaux vers le haut et vers le bas que l'on peut faire à partir de ce pixel en étant toujours sur un contour. Ce nombre nous donne la longueur du trait pour ce pixel particulier. En terme d'implémentation, il y a une petite subtilité tous les pixels de ce trait sont associés à ce même trait et donc à ce même nombre et il est bien inutile de recompter la longueur de ce trait. Il convient donc de retenir que ce trait a déjà été compté par exemple en le supprimant une fois compté.
 4. On se retrouve avec une succession de nombres correspondant à des longueurs de traits par position et par angle de rotation. Et on voudrait un unique nombre. Je propose de faire la moyenne de ces nombres (en additionnant pour chaque position et sans tenir compte des segments de taille nul) ce qui donne une courbe dépendant seulement de l'angle de rotation. Je propose d'en prendre la valeur maximale.

Cette idée s'adapte assez bien à la recherche des coins à angle droit, en modifiant l'étape 3, où on cherche le nombre de déplacements horizontaux et verticaux où simultanément on reste sur un contour.

La deuxième partie de la discussion est comment les valeurs associées à chacune des briques de base peuvent être combinées en une seule valeur. L'intuition qui me guide est que la façon de combiner est celle qui devrait donner les meilleurs résultats en terme d'OA vis-à-vis du ground truth lorsqu'on maximise cette valeur combinée pour choisir l'ensemble des valeurs des paramètres. Je m'attends à ce que si on faisait cette expérience pour une valeur donnée, l'OA aurait tendance à monter de façon irrégulière jusqu'à atteindre un maximum puis ensuite à décroître ou à avoir un comportement complètement aléatoire mais avec en moyenne des valeurs faible au delà de cette valeur optimale. On ne peut pas utiliser cette expérience pour déterminer cette valeur optimale (cela transformerait cette classification non-supervisée en une classification supervisée). Ce qui m'amène à penser qu'on aurait ce résultat est que dans la réalité la mesure associée à chaque est grande (ou petite) et il est très très rare d'obtenir une valeur aussi grande ou petite en tirant aléatoirement les valeurs des paramètres. Mais d'une part les valeurs de l'image hyperspectrale ne sont pas la vraie réalité, d'autre part la réalité n'est pas nette (par exemple la délimitation entre deux champs peut en fait correspondre à une route, une maison peut se trouver dans le champ, etc...) et l'algorithme fait des simplifications qui peuvent amener à faire un faux choix.

Ce que je propose est de chercher pour chaque brique à mesurer la proximité de cette valeur avec une valeur raisonnable (i.e. celle que l'on aurait avec un ground truth raisonnable). Cette proximité, je l'interprète comme la probabilité p que la classification soit correcte connaissant la mesure associée à cette brique. Par exemple on pourrait dire que si l'application à chaque pixel de la classification d'une modification de sa classe avec une probabilité q donne telle valeur de la mesure alors $p = 1 - q$. Enfin je suppose que les différentes briques mesurent des notions qui sont de natures différentes (mais est-ce vraiment le cas), et donc la mesure totale est le produit de ces différentes probabilités p associée à chaque brique. Concrètement il n'est pas simple d'implémenter réellement cette idée, une idée simple est d'utiliser un `ground truth` quelconque de faire une petite expérience et de tracer une courbe et d'en déduire graphiquement une échelle très approximative convertissant la mesure en une sorte de probabilité, et finalement de construire une fonction linéaire par morceaux pour faire cette conversion https://fr.wikipedia.org/wiki/Application_lin%C3%A9aire_par_morceaux. Mais il est possible aussi d'utiliser une régression non-linéaire qui est implémenté dans Python de manière très efficace https://fr.wikibooks.org/wiki/Python_pour_le_calcul_scientifique/R%C3%A9gression_et_optimisation. La difficulté en régression non-linéaire est qu'il faut se donner un ensemble de fonction de base et les fonctions linéaires par morceaux peuvent être cet ensemble de base si on ne demande pas au programme de chercher les valeurs seuils. Si on doit chercher ces valeurs seuils alors c'est une optimisation non-linéaire que doit faire le programme. Sinon c'est très simple et il suffit d'utiliser les fonctions $1_{[x_i, x_{i+1}]}(x)$ et $x1_{[x_i, x_{i+1}]}(x)$ pour le faire. J'ai eu l'occasion d'utiliser les réseaux de neurones pour faire de la régression non-linéaire 1D, c'est très efficace et finalement assez simple <https://lucidar.me/fr/neural-networks/curve-fitting-nonlinear-regression/>.

4 Informations supplémentaires

La discussion sur les propriétés théoriques de la `clustering_accuracy` du `compte_rendu_3_4` est discuté dans la section 4 du document écrit en novembre 2023. Cette section contient la phrase `Unfortunately, the lack of resemblance with the ground truth of this good OAC-performance only show that  $OA_c$  and  $AA_c$  seem not to be reliable metrics when they are below 0.7.` Je ne me souviens pas d'où vient le 0.7 mais je pense que c'est une idée à avoir en tête.

Le document mentionne aussi une pratique courante en Machine learning : `are the centered and normalized coordinates, (note that for the sake of clarity, the other features had also been centered and normalized). With this normalization, I and J can be seen as some kind of random variable following a uniform law on the interval  $[-\sqrt{3}, \sqrt{3}]$ .` Concrètement avant de mettre les valeurs des intensités $I_{m,n,k}$ dans un algorithme, il est généralement conseillé de calculer la moyenne et l'écart-type des valeurs des intensités pour chaque longueur d'onde k .

$$\mu_k = \frac{1}{MN} \sum_{m,n} I_{m,n,k} \text{ et } \sigma_k = \sqrt{\frac{1}{MN} \sum_{m,n} (I_{m,n,k} - \mu_k)^2} \quad (2)$$

Ensuite on remplace les valeurs de $I_{m,n,k}$ par

$$I'_{m,n,k} = \frac{I_{m,n,k} - \mu_k}{\sigma_k} \quad (3)$$

dont la moyenne et l'écart-type valent 0 et 1. C'est ce qu'on appelle centrer et normaliser. Ceci est expliqué dans [https://en.wikipedia.org/wiki/Normalization_\(machine_learning\)](https://en.wikipedia.org/wiki/Normalization_(machine_learning)) dans un contexte plus délicat celui de la renormalisation régulière au cours de l'apprentissage dans un réseau de neurones.

Dans les différentes équations, les traits verticaux qu'on utilise pour la valeur absolue ici compte le nombre de pixels contenus dans les ensembles de pixels. C'est ce même sens qui est donné à #.

Le document était à l'origine adressé à des chercheurs qui utilisent une technique appelée **Random Forest** qui est une technique de classification supervisée et j'avais à l'époque essayé d'adapter les idées à cette technique. C'est ce qui explique les sections 7.7 et 7.9, 7.2, 7.3, 7.4. En fait **Random Forest** a l'avantage de remplacer l'optimisation des valeurs de l'ensemble des paramètres par une moyenne de résultats de classification obtenues en étant guidé par cette mesure de vraisemblance. Le terme **imbalanced** est un terme très courant dans cette communauté scientifique qui signifie que le nombre de pixels de la classe ayant le plus de pixels est largement supérieur au nombre des pixels des classes ayant le moins grand nombre de pixels.

Dans le document, j'insiste sur le fait que la classification non-supervisée ne doit pas utiliser le **ground truth**, mais je pense que son utilisation ou le fait de chercher à avoir des représentations graphiques peut être très utile pour concrétiser et s'appropriier les idées.